

I am thinking improving a little bit our code structure. In the `construir_ala`, we did some modifications in our structure. I am thinking of taking all of the information and modify those informations in `generate_structure`. At the moment, I am removing the "inserted nodes and deletion of very close nodes" part of `construir_ala` and move those function in `generar_structura` function. I think this is good to have more modularity. I want the `construir_ala` to give me all the information needed and then in `generate_structure` is where I modify and do functionalities. If in the future, the `generate_structure` is more complex and long, then I will divide it into two parts. This way the `construir_ala` function is final and I only modify the parts that are needed to modify, to avoid doing what we are doing right now like the inserted nodes moving the functionalities to other parts. What do you think? " "I aim to establish a clear and standardized documentation framework to ensure accurate interpretation of the information I provide. This documentation will serve two main purposes: first, it will act as a reference guide for me when writing the thesis later, helping maintain consistency and clarity in the presentation of my ideas. Second, it will facilitate effective communication with you, ensuring that the context and intent of the documented information are fully understood without room for misinterpretation." "

ChatGPT said:

Improving Code Structure: Moving Functionality from `construir_ala` to `generate_structure`

Your plan is **a great step toward better modularity!**  Here's why I agree with your approach and how we can implement it efficiently:

Summary of Your Proposal

1. **`construir_ala` remains final** → It generates and provides **all necessary data** without modification.
2. **`generate_structure` handles modifications** →
 - Move functionalities like **inserting nodes** and **removing very close nodes** here.
 - Any **future structural changes** will only happen in `generate_structure`, keeping `construir_ala` untouched.
3. **If `generate_structure` becomes too complex, we split it into subfunctions.**

 **This approach ensures:**

- **Better separation of concerns** → `construir_ala` only generates, `generate_structure` modifies.
 - **Easier debugging and future modifications.**
 - **Avoids constantly moving code around**, as we are doing now.
-

Implementation Plan

Step 1: Modify `construir_ala`

- Ensure it only **creates and organizes raw data** without modifications.
- The output will contain **all nodes and elements (before any processing)**.

matlab

CopyEdit

```
function [nodes_2D, elements_2D] = construir_ala(parameters)

% CONSTRUIR_ALA Generates the initial 2D wing structure data.

%

% Outputs:

%   nodes_2D - Table of raw 2D nodes.

%   elements_2D - Table of raw 2D elements.

%

% This function only generates the data; modifications (like inserting nodes)
% will be handled in generate_structure.

%

% Author: Jess Bern Amora Ycong

% Date: 29-Jan-2025


% Generate raw nodes and elements (unchanged)

nodes_2D = generate_wing_nodes(parameters);

elements_2D = generate_wing_elements(nodes_2D);
```

```
% No modification happens here!  
end
```

◆ Step 2: Move Processing to generate_structure

- **Move** the "inserted nodes" and "removal of close nodes" from construir_ala to generate_structure.
- If generate_structure grows too large, we split it into **subfunctions**.

matlab

CopyEdit

```
function [nodes_3D, elements_3D] = generate_structure(nodes_2D, elements_2D,  
H)
```

```
% GENERATE_STRUCTURE Processes raw 2D wing data into 3D, applies  
modifications.
```

```
%
```

```
% Inputs:
```

```
%   nodes_2D   - Table of 2D nodes (from construir_ala).
```

```
%   elements_2D - Table of 2D elements (from construir_ala).
```

```
%   H         - Wing thickness.
```

```
%
```

```
% Outputs:
```

```
%   nodes_3D   - Modified and structured 3D nodes.
```

```
%   elements_3D - Modified 3D elements.
```

```
%
```

```
% Author: Jess Bern Amora Ycong
```

```
% Date: 29-Jan-2025
```

```
% --- Step 1: Insert additional nodes (moved from construir_ala) ---
```

```
nodes_2D = insert_missing_nodes(nodes_2D);
```

```
% --- Step 2: Remove very close nodes (moved from construir_ala) ---
nodes_2D = remove_close_nodes(nodes_2D);

% --- Step 3: Generate 3D nodes ---
nodes_3D = generate_3D_nodes(nodes_2D, H);

% --- Step 4: Process 3D elements ---
elements_3D = preprocess_3D_quads(elements_2D);

end
```

✚ Step 3: Subdivide generate_structure (If Needed)

If generate_structure **gets too long**, we can **divide** it into:

1. generate_3D_nodes.m – Creates **3D node data**.
2. preprocess_3D_quads.m – Converts **2D quads to 3D**.
3. clean_nodes.m – Handles **removing duplicates or close nodes**.

✅ This ensures we don't create **one giant function** that becomes difficult to manage.

✚ Benefits of This Approach

✓ Better Modularity

- **Each function has a clear responsibility:**
 - construir_ala → Generates **raw** wing data.
 - generate_structure → **Modifies** and **processes** data.
 - Subfunctions handle **specific tasks** (e.g., removing close nodes, inserting nodes).

✓ Easier to Debug

- If there's an issue with **node processing**, we only check generate_structure.m.

- If we modify elements, we update **only** preprocess_3D_quads.m.

✓ Future-Proofing

- If future changes are needed (e.g., **new wing modifications**), we **only modify generate_structure**, not construir_ala.
- Keeps construir_ala **final and stable**.

📌 How to Document This Change?

Since you're establishing a **standardized documentation framework**, we should **update our documentation** accordingly:

📄 Update: Function Responsibilities

Function	Responsibility
construir_ala.m	Generates raw wing structure (nodes & elements).
generate_structure.m	Processes & modifies wing structure (inserts nodes, removes close nodes, 3D conversion).
generate_3D_nodes.m	Converts 2D nodes into 3D .
preprocess_3D_quads.m	Converts 2D quads into 3D (extrados & intrados).
clean_nodes.m	Removes duplicate/close nodes from the structure.

📌 Next Steps

1. ☒ **Modify construir_ala** to stop modifying data.
2. ☒ **Move insert/remove node functionalities** to generate_structure.
3. ☒ **Test the new structure** to ensure correctness.
4. ☒ **Update documentation** to reflect new modular design.